CareFlow: AI-Assisted Hospital Patient Flow Management with Live Queue and Resource Coordination

A. Ramu, B. Rushi
23K91A6715, 23K91A6720
Department of CSE (Data Science)
[COLLEGE NAME], [CITY]
[EMAIL]

B. Raj Kumar, A. Prabhas
23K91A6725, 23K91A6703
Department of CSE (Data Science)
[COLLEGE NAME], [CITY]
[EMAIL]

[GUIDE NAME]
[DESIGNATION]
Department of CSE (Data Science)
[COLLEGE NAME], [CITY]
[EMAIL]

***Abstract*—**Hospitals depend on coordinated transitions among registration, outpatient queues, consultation, laboratories, wards and intensive care. This paper presents CareFlow, a role-based web prototype that creates digital tokens, records visit movement, exposes laboratory and bed state, publishes live updates and applies two machine-learning models for triage priority and waiting-time estimation. The implementation uses React, FastAPI, SQLAlchemy, SQLite, JWT and WebSockets. The models are evaluated only on deterministic synthetic generators: the wait-time Random Forest obtained R-squared 0.951041, MAE 5.982 minutes and RMSE 8.580 minutes, while the Gradient Boosting triage classifier obtained 96.80% accuracy and 0.898475 macro F1. Critical-class recall was 63%, which prevents a clinical-validity claim. In a local sequential smoke run, 150 of 150 read requests returned HTTP 200. The contribution is an integrated and measurable patient-flow prototype with explicit evidence boundaries, not an autonomous clinical system.

***Index Terms*—**bed management, hospital operations, patient flow, queue prediction, real-time dashboard, triage, WebSocket

I. INTRODUCTION

Patient flow is an operational chain rather than a single appointment. A visit may begin at reception, wait in an outpatient or emergency queue, enter consultation, require laboratory work, move to a ward or intensive-care bed and finish at discharge. When those transitions are maintained in telephone calls, registers or disconnected screens, staff cannot see a reliable current state and patients receive uncertain queue information.

CareFlow addresses this coordination problem with one persisted Visit state and role-specific interfaces. Reception creates the visit and token; doctors manage consultation and routing; laboratory staff progress tests; nurses manage bed state; administrators see aggregate load; and a public display exposes now-serving and upcoming tokens. Two estimators provide decision support but do not replace staff judgement.

The contributions are: (1) priority-aware digital tokens linked to one visit identity; (2) integrated laboratory, ward and ICU workflow; (3) live role views using REST and WebSocket events; and (4) a reproducible evidence boundary that separates synthetic-model metrics, local software measurements and unmeasured clinical impact. Section II reviews related work, Section III presents the method, Section IV reports results and Section V concludes.

The scope is deliberately operational. CareFlow does not diagnose disease, recommend treatment, replace a validated emergency scoring protocol or claim interoperability with an installed hospital information system. Instead, it demonstrates how a shared visit identifier can connect actions that are frequently represented in separate lists. The distinction matters because a visually complete dashboard can otherwise be mistaken for clinical evidence. Every quantitative statement in this paper is therefore tagged by its evidence source: deterministic synthetic model evaluation, a populated demonstration database, or a local software smoke measurement.

A. System Hardware

Measurements used Windows 11 Pro 64-bit on a 12th Gen Intel Core i5-12500 with six physical cores, twelve logical processors and 15.7 GB RAM. Python 3.11.9 executed the model evaluation; Chrome at 1440 x 900 produced the screen evidence.

B. Software Requirements

The backend uses FastAPI 0.115.0, SQLAlchemy 2.0.35, scikit-learn 1.5.2, Joblib and SQLite. The frontend uses React 19, TypeScript, Vite, Axios, Zustand, Recharts and Tailwind

CSS. JWT protects staff routes and an in-process WebSocket manager publishes changes.

II. LITERATURE SURVEY

The Alleviate base paper integrates appointments, prescriptions, clinic dashboards, doctor verification, maps and personalised diet/exercise support [1]. Its logistic-regression recommendation task reports 97.24% precision and its diet model 84.76% precision. Those tasks differ from hospital flow, so their figures are contextual rather than numerical baselines.

Access-oriented systems combine symptom assessment, records and hospital recommendation [2], locate nearby facilities [3]–[5], or predict disease and suggest doctors [6], [9], [16]. Personalisation studies add monitoring, diet or exercise [10], [15], while conversational systems use NLP and chatbots [11]–[14]. These systems improve access or advice but do not necessarily represent the operational state of a person moving among queues, tests and beds.

Recent waiting-time studies motivate regression and interpretable prediction [17]–[19]. Their institutions, features and outcomes differ from this project. CareFlow therefore reports its wait model only as recovery of a designed synthetic process. The research gap is an integrated, auditable application that links priority, token order, department movement, laboratory work, capacity and live role views while exposing the limits of its evidence.

The literature also highlights a recurring evaluation hazard. A high score on a recommendation, diagnosis or waiting-time task cannot be transferred to another hospital, target variable or patient population. CareFlow consequently avoids a numerical league table. The cited work establishes design motivation—integrated access, personalisation, conversational support and delay prediction—whereas the prototype contribution is the explicit linkage of operational states and the reproducible reporting of what has and has not been measured.

TABLE I

RELATED WORK AND RESEARCH GAP

Work	Approach	Data/result	Limitation
Alleviate [1]	Integrated mobile AI	97.24% recommendation precision	Different task
Access systems [2]–[6]	Diagnosis/location	Source-specific	No internal flow
Personalisation [10],[15]	Diet/exercise	User metrics	No capacity state
Waiting studies [17]–[19]	ML prediction	Observed outpatient settings	Not shared locally
CareFlow	RF + GB + workflow	Synthetic tests	No clinical validation

III. METHODOLOGY

The implemented architecture has five concerns: React interfaces, FastAPI routes, SQLAlchemy persistence, a Joblib model layer and WebSocket publication. JWT decoding resolves the current user and role dependencies guard write routes. Seven entities—User, Department, Patient, Visit, Bed, LabTest and MovementLog—represent staff, operational state and traceability.

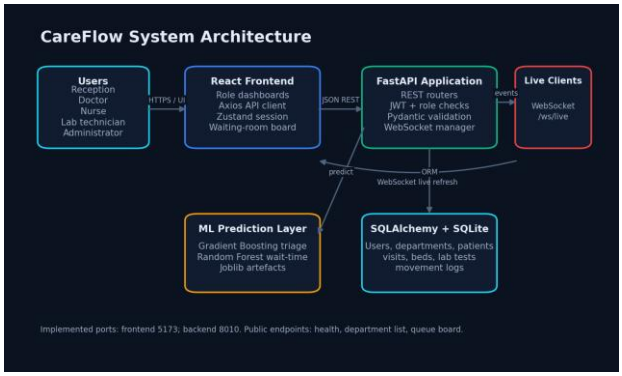


Fig. 1. Implemented CareFlow architecture.

At registration, the API validates the department, matches a patient by phone or creates an MRN, completes missing individual vital values with documented defaults, predicts priority, allocates the next token, counts higher-ranked waiting visits, predicts expected wait, persists Visit and MovementLog, commits and broadcasts visit_created. Queue order is critical, urgent and normal, then token number.

A. State and Role Design

Visit is the central operational record. It contains the patient and department links, token, priority, predicted wait, current status and timing fields. MovementLog records transitions; LabTest records an order independently of its display state; and Bed represents General Ward or ICU capacity. Reception may register visits, doctors may start or complete consultation and route the patient, laboratory staff may progress tests, nurses may allocate or release beds, and administrators may view system-wide summaries. This separation reduces accidental cross-role writes while retaining a common read model.

The frontend requests REST resources for stable lists and subscribes to WebSocket messages for change notification. On receipt, the relevant view refreshes its data rather than trusting the event as the sole source of truth. The approach is simple and suitable for a single-process prototype, but it does not provide durable delivery, replay or multi-instance fan-out. Those properties would require a message broker, authenticated channel subscriptions and an event contract with versioning.

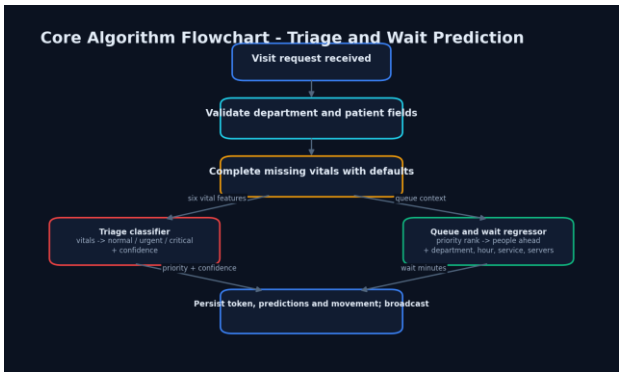


Fig. 2. Core triage and wait-time flow.

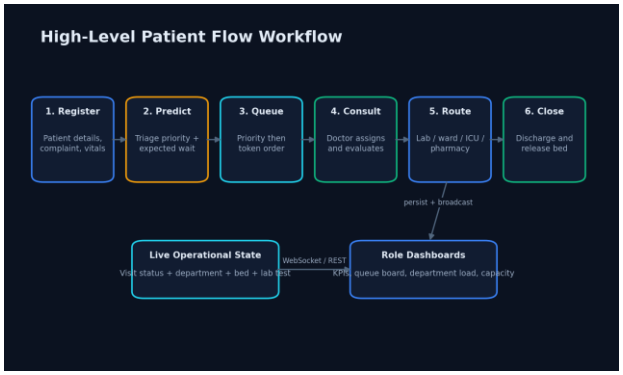


Fig. 3. Cross-role patient-flow state transitions.

B. Synthetic Data and Triage Model

The triage generator creates 15,000 rows of age, heart rate, systolic pressure, SpO2, temperature and respiratory rate. A simplified early-warning function assigns normal for scores 0–3, urgent for 4–6 and critical for 7 or more. An 85/15 stratified split produces 12,750 training and 2,250 test rows. GradientBoostingClassifier uses 150 estimators, maximum depth three and random state 42.

C. Wait-Time Model

The wait generator creates 12,000 rows with department type, queue ahead, UTC hour, priority, average service minutes and server count. The target approximates (queue/servers) times service duration, multiplied by rush and priority factors, plus Gaussian noise, clipped to 0–240 minutes. An 85/15 split gives 10,200 training and 1,800 test rows. RandomForestRegressor uses 200 trees and maximum depth ten.

Both generators use NumPy seed 42. Encoders and feature order are stored with the estimators. The datasets contain no real patient data and have no external dataset licence. This design supports reproducibility, not clinical generalisation.

D. Measurement Protocol

The evaluation regenerated each dataset from the recorded seed and applied the stored 85/15 split policy. Regression was assessed with coefficient of determination, mean absolute error and root mean squared error. Classification was assessed with accuracy and macro-averaged precision, recall and F1, followed by the complete three-class confusion matrix. Macro measures were included because normal, urgent and critical classes are not equally frequent. Critical recall was treated as a safety-relevant diagnostic rather than hidden behind aggregate accuracy.

Software observations were collected separately. Ten read endpoints were requested 15 times each over loopback, yielding 150 responses, and WebSocket connection establishment was timed once. The frontend was checked with its configured lint and production-build commands. These are engineering smoke checks, not concurrent load tests: they exclude network distance, browser concurrency, write contention, warm-up analysis, confidence intervals, database growth and production infrastructure.

E. Reproducibility and Operational Invariants

Reproducibility rests on four recorded elements: fixed random seeds, explicit generator formulas, fixed estimator hyperparameters and saved preprocessing metadata. The model artefacts retain the label encoders and feature order required at inference time. The application facts file records the evaluated environment, sample counts, class order and exact metrics. This permits another evaluator to distinguish a regenerated experiment from a screenshot value and prevents a model score from being silently relabelled as a hospital outcome.

Several invariants are required for operational correctness. A token must be unique within its allocation scope; priority ordering must precede token ordering; a bed cannot be allocated to two active visits; discharge must release the associated resource; laboratory completion must not imply visit completion unless a separate transition occurs; and movement history must preserve who changed a state and when. The prototype implements portions of these rules through route logic and relational fields, but it does not yet enforce all of them with database constraints and automated concurrent tests.

Security was reviewed as implemented rather than assumed from framework choice. Staff HTTP routes use JWT-derived identity and role dependencies, passwords are represented through the authentication layer, and the frontend separates role views. Remaining production gaps include a development secret that must be replaced and rotated, missing authenticated WebSocket subscription, incomplete audit coverage, no documented rate limiting, and no formal privacy or retention policy. Consequently, the demonstration must not contain identifiable real patient records.

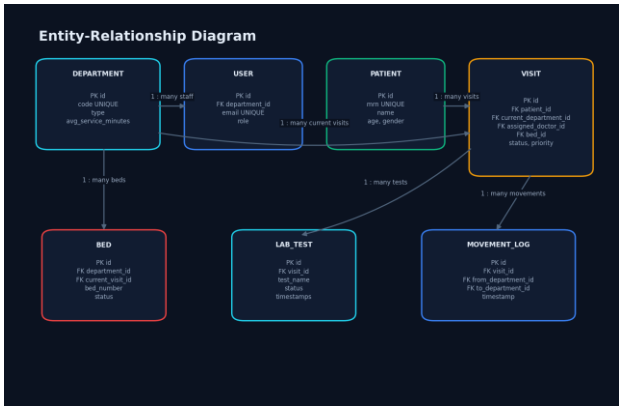


Fig. 4. Core data entities and relationships.

IV. RESULTS

The wait model achieved R-squared 0.951041, MAE 5.982 minutes and RMSE 8.580 minutes. The triage classifier achieved 0.9680 accuracy, 0.962435 macro precision, 0.858574 macro recall and 0.898475 macro F1. Of 100 actual critical test examples, 63 were classified critical and 37 urgent. This 63% critical recall is more decision-relevant than the overall accuracy.

TABLE II

MEASURED MODEL AND SOFTWARE RESULTS

Measure	Value	Condition
Wait R-squared	0.951041	1,800 synthetic test rows
Wait MAE / RMSE	5.982 / 8.580 min	Generator-matched test
Triage accuracy / macro F1	0.9680 / 0.898475	2,250 synthetic test rows
Critical recall	0.63	100 critical test cases
HTTP smoke	150/150 status 200	Sequential loopback reads
Mean endpoint range	3.985–14.231 ms	15 requests per endpoint



Fig. 5. Measured model scores.

A running synthetic demonstration database contained seven users, seven departments, 23 patients and visits, 18 beds and five laboratory tests. The administrator snapshot showed 23 patients for the day, 18 active visits, 11.1 minutes average predicted wait, two active critical cases, 61% overall bed occupancy and 67% ICU occupancy. These are demonstration values, not hospital-study results.

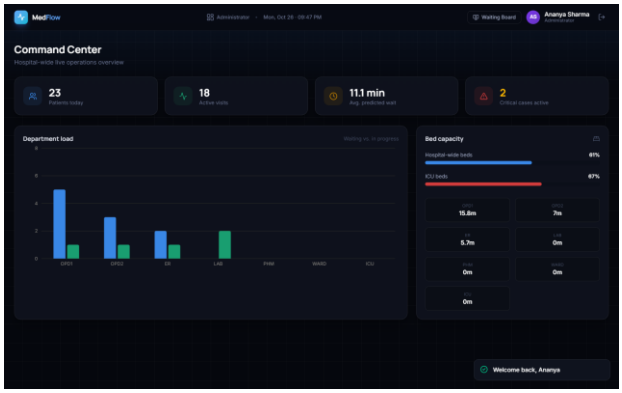


Fig. 6. Populated administrator dashboard.

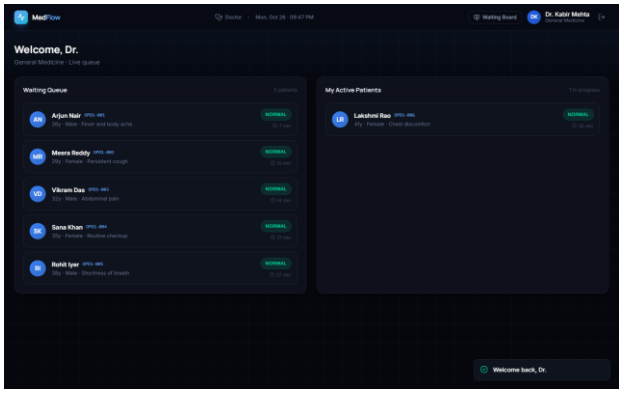


Fig. 7. Doctor queue and active patient view.

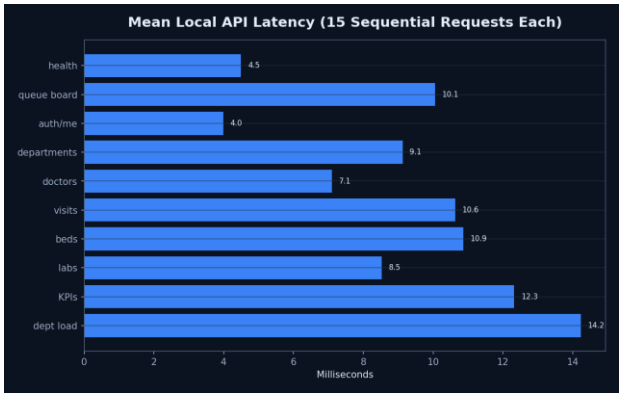


Fig. 8. Sequential loopback endpoint latency.

Frontend lint exited zero with 14 warnings and the production build exited zero after transforming 3,573 modules. The browser capture reported no console errors. Observed failures include a nurse tab that requires explicit selection on first load, incomplete coupling of laboratory and

visit states, seeded occupied beds without linked visits, and two large JavaScript chunks.

The endpoint means ranged from 3.985 to 14.231 ms in the sequential loopback run, and the measured WebSocket connection took 46.657 ms. The variation reflects endpoint work and local scheduling, but the protocol is too small to support a service-level objective. Likewise, 150 successful reads establish only that the selected routes responded in this environment; they do not establish authorization completeness, transaction safety under competing writes or resilience after process failure.

A. Error Analysis and Validity

The triage confusion matrix, ordered critical, normal and urgent, was [[63, 0, 37], [0, 1525, 2], [2, 31, 590]]. Most errors therefore involved adjacent urgency boundaries, but 37 critical examples were reduced to urgent. Even though no critical example was predicted normal, the missed critical decisions are sufficient to reject autonomous use. Future evaluation must specify the clinical cost of each transition, establish thresholds with clinicians and test calibration and subgroup performance on governed real data.

Internal validity is limited by formula-derived labels and generator-matched train and test distributions. External validity is absent because the records are synthetic and originate from no hospital. Construct validity is also limited: the target represents a simplified waiting process and the early-warning function is not a validated triage protocol. Software validity is limited by one workstation, one browser viewport, sequential requests and no automated end-to-end test suite. These limitations are part of the result, not footnotes to be removed from it.

B. Capacity and Workflow Observations

The demonstration bed inventory comprised 18 resources across ward and ICU categories. Overall occupancy was 61%, while ICU occupancy was 67% in the captured state. These figures verify that the dashboard can aggregate seeded resource records and distinguish categories; they do not estimate utilisation for a hospital. Five laboratory tests exercised pending and progressed views, and 23 visits provided non-empty queue, consultation and dashboard states. The records were intentionally synthetic and remain unsuitable for epidemiological or operational inference.

Visual walkthroughs covered the public landing page, staff login, public waiting board, administrator summary, reception registration, doctor queue, patient detail, laboratory ordering, admission, ward, ICU and laboratory views. This coverage establishes that the major screens could be reached and populated in the recorded environment. It is not a substitute for scripted acceptance testing: transition assertions, permission-denial cases, refresh consistency, keyboard accessibility, responsive layouts and recovery after interrupted writes remain to be tested.

The observed nurse first-load issue illustrates the value of testing workflow state rather than only route availability. A page may load successfully while its default tab does not expose the expected data until the user selects a ward explicitly. Similarly, a laboratory record can progress without proving that every corresponding visit transition is synchronised. These findings motivate end-to-end scenarios that begin with registration and assert the final patient, visit, test, movement and bed records together.

Comparison with Alleviate is qualitative because datasets and tasks differ. Alleviate emphasises patient access and recommendation; CareFlow emphasises operational flow and shared resource state. No measured reduction in real waiting time, clinical accuracy, deployment availability or user acceptance is claimed.

V. CONCLUSION

CareFlow demonstrates that a compact full-stack system can unify digital tokens, priority-aware queues, movement, laboratory work, bed/ICU visibility and live role dashboards. The local software checks succeeded and the saved models produced reproducible synthetic-test results. However, critical recall, synthetic data and absent clinical evaluation prevent clinical use.

The practical lesson is that integration and evidence should advance together. A system may correctly expose a queue and still require transition invariants, authorization tests and failure recovery; a model may score highly on generated samples and still be unsafe for a patient population. Retaining these distinctions makes the prototype a useful engineering baseline for supervised research rather than an overstated clinical product.

A staged validation path is proposed. First, automated unit and API tests should lock down token allocation, state transitions, authorization and resource release. Second, concurrent synthetic scenarios should measure database and event consistency. Third, clinicians should review the feature definitions, error costs and user interface using only non-identifiable cases. Only after governance approval should a de-identified retrospective dataset be used for temporal and external model evaluation, calibration and subgroup analysis. Prospective deployment would then require monitoring, rollback criteria and human override.

Future work should obtain governed de-identified flow data, perform temporal and external validation, optimise critical recall, calibrate uncertainty, add automated state-transition tests and conduct clinician-led usability assessment. Production engineering should replace SQLite and in-process broadcasts, authenticate live channels, rotate secrets, add audit and privacy controls, and integrate hospital systems through HL7 FHIR.

REFERENCES

[1] S. A. N. Saritha, G. Kalnoor, P. R. Bhat, P. A. Rao, P. Prajwal, and P. T. Pradeep, ‘Alleviate: An AI-Powered Mobile Application for Personalized Healthcare Management and Doctor Recommendation,’ IEEE Access, vol. 13, pp. 215822–215832, 2025, doi: 10.1109/ACCESS.2025.3647089.

[2] S.-K. Tan, S.-C. Chong, K.-K. Wee, and L.-Y. Chong, ‘Personalized healthcare: A comprehensive approach for symptom diagnosis and

hospital recommendations using AI and location services,' Journal of Informatics and Web Engineering, vol. 3, no. 1, pp. 117–135, 2024.

[3] M. Abinaya and R. Ganesan, 'Effective search mechanism for finding nearest healthcare facilities,' in Proc. Global Conference on Communication Technologies, 2015, pp. 534–538.

[4] M. W. Munir, S. M. Omair, and M. Z. U. Haque, 'An Android based application for determine a specialized hospital nearest to patient's location,' International Journal of Computer Applications, vol. 118, no. 9, pp. 43–46, 2015.

[5] R. A. Nimbalkar and R. A. Fadnavis, 'Domain specific search of nearest hospital and healthcare management system,' in Proc. Recent Advances in Engineering and Computational Sciences, 2014, pp. 1–5.

[6] A. Kumar, 'Disease prediction and doctor recommendation system using machine learning approaches,' International Journal for Research in Applied Science and Engineering Technology, vol. 9, no. 7, pp. 34–44, 2021.

[7] L. Bjorkdahl, O. Pauli, J. Ostman, C. Ceccobello, S. Lundell, and M. Kjellberg, 'Towards holistic disease risk prediction using small language models,' arXiv:2408.06943, 2024.

[8] R. Umbare, R. Patil, T. Mukund, A. Rathod, and P. Rajurkar, 'A smart healthcare application using artificial intelligence and machine learning,' in Proc. ICIPCN, 2024, pp. 204–210.

[9] D. Gujar, R. Biyani, T. Bramhane, S. Bhosale, and T. P. Vaidya, 'Disease prediction and doctor recommendation system,' International Research Journal of Engineering and Technology, vol. 5, pp. 3207–3209, 2018.

[10] D. Mogaveera, V. Mathur, and S. Waghela, 'E-health monitoring system with diet and fitness recommendation using machine learning,' in Proc. ICICT, 2021, pp. 694–700.

[11] P. Tanwar, K. Bansal, A. Sharma, and N. Dagar, 'AI based chatbot for healthcare using machine learning,' in Proc. ICAICCIT, 2023, pp. 751–755.

[12] R. Surve, T. Purohit, R. Joseph, and P. Shaikh, 'Healthcare chatbot using machine learning and NLP,' in Proc. ICAST, 2023, pp. 411–416.

[13] H. Mendapara, S. Digole, M. Thakur, and A. Dange, 'AI based healthcare chatbot system using natural language processing,' International Journal of Scientific Research and Engineering Development, vol. 4, no. 2, pp. 89–96, 2021.

[14] M. M. Rahman et al., 'Web-based multiple disease prediction system with transfer learning and healthcare chatbot,' in Proc. IEEE ICMI, 2024, pp. 1–5.

[15] J. Tseng et al., 'An interactive healthcare system with personalized diet and exercise guideline recommendation,' in Proc. TAAI, 2015, pp. 525–532.

[16] T. Bhanuteja, K. V. N. Kumar, K. S. Poornachand, C. C. V. N. Ashish, and P. Anudeep, 'Symptoms based multiple disease prediction model using machine learning approach,' IJITEE, vol. 10, no. 9, pp. 67–72, 2021.

[17] 'Waiting Time Prediction in Queue Management: Leveraging Machine Learning Approach,' in Proc. International Conference on Recent Progresses in Science, Engineering and Technology, 2024.

[18] 'Prediction of outpatient waiting time using machine learning in a tertiary children's hospital,' Translational Pediatrics, 2023.

[19] 'Dissatisfaction-considered waiting time prediction for outpatients with interpretable machine learning,' Health Care Management Science, 2024.